An Architecture to Automate Performance Tests on Microservices

André de Camargo
Graduate Program in Computer Science
Federal Univ. of Santa Catarina – Florianópolis, Brazil
IBM – São Paulo, Brazil
asdcamargo@gmail.com

Ivan Salvadori
Graduate Program in Computer Science
Federal University of Santa Catarina
Florianópolis, Brazil
ivanlsalvadori@gmail.com

Ronaldo dos Santos Mello
Department of Informatics and Statistics
Federal University of Santa Catarina
Florianópolis, Brazil
r.mello@ufsc.br

Frank Siqueira
Department of Informatics and Statistics
Federal University of Santa Catarina
Florianópolis, Brazil
frank.siqueira@ufsc.br

ABSTRACT
The microservices architecture provides a new approach to develop applications. As opposed to monolithic applications, in which the application comprises a single software artifact, an application based on the microservices architecture is composed by a set of services, each one designed to perform a single and well-defined task. These services allow the development team to decouple several parts of the application using different frameworks, languages and hardware for each part of the system. One of the drawbacks for adopting the microservices architecture to develop applications is testability. In a single application test boundaries can be more easily established and tend to be more stable as the application evolves, while with microservices we can have a set of hundreds of services that operate together and are prone to change more rapidly. Each one of these services needs to be tested and updated as the service changes. In addition, the different characteristics of these services such as languages, frameworks or the used infrastructure have to be considered in the testing phase. Performance tests are applied to assure that a particular software complies with a set of non-functional requirements such as throughput and response time. These metrics are important to ensure that business constraints are respected and to help finding performance bottlenecks. In this paper, we present a new approach to allow the performance tests to be executed in an automated way, with each microservice providing a test specification that is used to perform tests. Along with the architecture, we also provide a framework that implements some key concepts of this architecture. This framework is available as an open source project[1].

CCS Concepts
• Software and its engineering→Software organization and properties→Software system structures →Distributed systems organizing principles • Computer systems organization →Architectures →Distributed architectures • Software and its engineering →Software organization and properties →Extra-functional properties →Software performance

Keywords
Microservices; test automation; performance test.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
iiWAS '16, November 28-30, 2016, Singapore, Singapore
© 2016 ACM. ISBN 978-1-4503-4807-2/16/11$15.00
DOI: http://dx.doi.org/10.1145/3011141.3011179

1. INTRODUCTION
The microservices architecture has grown in popularity in the past few years due to its capacity to overcome some of the major problems found in monolithic applications [4]. The key concept regarding microservices is to develop an application as a set of small services, with each service implementing a single feature, running in its own process, infrastructure and programming language.

The concept of microservices is based on a well-known principle adopted in software engineering. The idea of separating an application into more manageable components is present in several approaches for software design, such as SOA (Service Oriented Architecture) [10, 11]. Although, we can observe that SOA tries to deal with integration between applications. On the other hand, the microservices architecture is a way of organizing a single application. The main difference is in the granularity level, i.e., microservices are fine-grained services designed to provide specific unique features, while on SOA features are provided through services kept under the same structure, deployed as a single unit and running under the same infrastructure.

As applications have grown in size and complexity, the development time and cost have also increased [4]. Besides that, we can notice some major difficulties to scale, synchronize changes, deliver new features and replace frameworks or libraries [12]. This leads to the search for a new way to mitigate these problems. In this scenario, the microservices architecture arises as a possible solution for overcoming these difficulties.

However, microservices are not the silver bullet for all problems found in monolithic applications [13]. Although this architecture can overcome some of the difficulties faced with monolithic

[1] https://github.com/asdcamargo/fpts

applications, it also ended up raising some challenges such as [2, 13, 15]: orchestration complexity in the deployment of multiples services, managing and controlling the tests for each service, maturity of the team to coordinate changes, to define the service boundaries, to control shared libraries and code reuse.

Test challenges for adopting the microservices architecture include the same ones found in distributed systems in general: inter-service communication, coordination between services and distributed transactions [2]. In fact, microservices bring another challenge, which is to properly split the application in several components [4]. This means that we now have to establish the test boundaries of each service, which will have a set of tests to evaluate its compliance with the existing requirements. Moreover, this set of tests may involve other services, which imposes a challenge to track the test dependencies within each microservice.

We can quote the Gilt Group, which moved their online sales platform from a monolithic architecture to a set of microservices. After migration, they ended up with 156 microservices [8]. Testing each of these services separately is a hard task. Moreover, to keep each test specification up-to-date with the constant service changes is also difficult. The same challenge also applies for performance tests, each service may run in its own infrastructure with different frameworks and programming languages. By knowing each service performance parameters, the proper infrastructure can be provided to reduce costs. In addition, bottlenecks can be identified and corrected to improve the performance of each server individually. Innovative approaches to improve tests on microservices are demanded in this scenario.

The work done so far in microservices tests includes the development of a repository to centralize acceptance tests that use Behavior Driven Development (BDD) [1]. Another study was made to evaluate the performance of the hardware when using different types of containers to run microservices [11]. Regarding performance test in REST (Representational State Transfer) Web Services, [15] presents a framework to cover the different phases in the performance test execution. None of these works focuses on performance tests in a microservices architecture. In addition, on [21] a study was made to evaluate the most recent contributions on microservices, and 3 studies out of 23 focus on test improvements for microservices. All three contributions deal with test automation, but none focuses on performance tests in particular.

For the purpose of this work, we are considering end-to-end performance tests. This means that a remote client sends the requests over the network to the target service.

The architecture proposed in this paper is designed to overcome the test challenges in microservices, more precisely performance tests. As concerns to performance tests, it is extremely important to evaluate the performance that each service can deliver, individually. This data will dictate the amount of load that a service can handle, the resources spent on infrastructure and will help to find bottlenecks and to control the impact of changes on the performance. Our goal is to automate the execution of performance tests by attaching a specification that contains the test parameters to each service. Besides the test specification, the test application must be configured with parameters such as the amount of threads and requests, load time, test duration, and so on. We understand that these parameters can be easily defined and reused between tests; however, the test specification tends to be unique for each operation of a service. That being said, in this paper we will focus on the test specification and how it can be provided by the service to the test application.

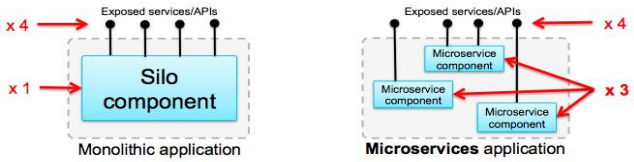

Figure 1. Microservices Application Exposing the Same Interfaces as the Monolithic Application [9].

The remainder of this paper is organized as follows. In section 2 we present the background on microservices and software testing, covering performance tests in more detail. Section 3 describes the related work and identifies gaps for improvement in the present scenario. Section 4 introduces the architecture for automating the performance tests of microservices. Section 5 presents a framework that was implemented to show some features of the architecture. Section 6 shows an evaluation of how the framework affects the overall performance of an application. Finally, section 7 presents the conclusions and perspectives for future work.

2. BACKGROUND

2.1 Microservices

The microservices architecture refers to an approach for developing applications as a set of small loosely coupled services that uses a lightweight network protocol to communicate with each other [4]. The approach is targeted to the internal organization of an application not the application external boundaries such as its communication with other applications or services.

This approach allow breaking the application complexity into a set of services, so that a change can affect only one component and not the whole application itself. Besides that, each service can be provided considering its hardware constraints using only the necessary resources and using the best technology for each purpose, leading to cost reduction with development and infrastructure [12].

Considering a web services based application, moving from a monolithic application to a microservices architecture will affect only the internal structure of the application. The exposed services will remain as they were in the monolithic application [9]. Therefore, external applications/APIs do not need to change after the migration from monolithic to microservices.

Some key features of using a microservices architecture are [12]:

- *Technology Heterogeneity:* With the application composed as a set of independent and loosely coupled services we have the freedom to pick the best tool that fullfill the needs. That can be the full application stack, from the programming language to the application server. Also, since services are small we can replace a technology with a low risk.
- *Resilience:* The key concept of resilience is to isolate failures so it doesn't affect the whole system. The natural independence of the services in a microservices architecture allow to isolate key services in its own infrastructure to keep them working even with failures in other services.
- *Scalability:* Each service boundary is well defined and organized around the business capabilities. This sort of granularity allow to scale each one of the services as needed, reducing costs and providing only the necessary resources as oposed to monolithic applications where we have to scale the whole application.
- *Ease of Deployment:* In a microservices architecture we can change a single service and deploy it independently in a short time.

This allow the changes to be developed and tested faster and with lower risk when compared to monolithic applications where even small changes requires the whole application to be deployed. Besides that, deployment problems can be easily isolated in a microservices architecture allowing fast rollback and the service restoration.

2.2 Software Testing

Software testing is the process of executing a program with the intent of finding errors [16]. Software testing is mainly divided into two methods of tests: white-box and black-box [16]. Black-box method consists of disregard the system internal behavior, instead, the test is made considering only the input and output data. On the other hand, White-box method consider the system internal structure causing each statement of the program to be executed at least once. In software testing area there are different test levels or stages [19, 20]:

- *Unit testing:* The objective of this phase is to test each component or software unit individually and independently without considering other parts of the application.
- *Integration testing:* This level focus on testing if the components work well after they are integrated. For this test, the application components are bound together and assembled in a single artifact.
- *System testing:* This level deals with the end-to-end test. The objective of this test is to use the requirements that were raised during the analysis and check if all requirements are being attended by the application.
- *Acceptance testing:* At this level, final users perform the test in the system to check if the solution delivered meets their needs.

In this context, performance tests are located in black-box method and system testing level. In addition, performance tests applied to web application are end-to-end tests, this means that the test requests are send over the network to the target server.

Considering performance testing the purpose of the test is to find out if the software complies with a set of non-functional requirements like throughput, response time and availability. In fact, the term performance testing is a generic term that may refer a set of different types of performance-related testing [17, 18] such as load test, stress test and capacity test.

Therefore, using the term performance test we are actually referring to any test that measures the stability, performance, scalability and throughput of a web application [18]. For the purpose of this work, we will use the term performance test once the present study can be used in any of its types of test.

3. RELATED WORK

Regarding acceptance tests, Rahman et al. [1] present an approach to be used for agile processes with BDD. This work tackles the specific problem of reuse of the steps in test scenarios. The contribution of this work is an architecture to centralize the acceptance tests of each service in one repository that can be shared across multiple services. The focus of this work is acceptance tests when using BDD; however, we are focusing on performance tests.

A study on microservice performance from the hardware perspective was presented by Amaral et al. [11]. The main goal of this paper was to analyze the difference, from the performance perspective, when using two types of containers to run microservices: master-slave and nested-container. On this work, a benchmark was used to measure CPU and network usage. The authors conclude that the nested-container is a more suitable model due to improved resource sharing. We claim that our main objective with our work is the performance of the application instead of the hardware.

Kao et al. [15] present a mechanism to execute performance tests on RESTful web applications. In this particular paper, the focus is on covering the whole process to execute performance tests: test case design, test script generation and test execution. We claim that this approach is too focused on the tester side and requires some knowledge of the test procedures. In addition, our goal is to allow test automation, without human intervention. In this scenario the main problem is to manage many test specifications (one for each operation), things like workload and test duration can be easily customized in a test application such as JMeter [14].

Our search for papers on the field of performance test for microservices have shown that there is a gap for improvements in this area. None of the works found is directly related to performance tests on microservices. Besides that, tests on microservices is described as one of the main challenges of this architecture [2, 13].

Performance tests on microservices are even more relevant considering the amount of services that may compose an application, which can easily go up to hundreds of services [8]. We have to consider that is necessary to know the performance of each service, individually. This way we can provide the necessary resources for the infrastructure find bottlenecks in the application and fulfill QoS requirements such as response time and throughput. It can be hard to control each of these services tests. Each service operation must be associated with a set of parameters to be used in the performance test. Therefore, a mean to automate the generation of these parameters will allow the execution of the performance tests in a seamless and decoupled way.

4. THE ARCHITECTURE

The architecture is designed to provide a clean and easy way to evaluate the performance of microservices. Our solution is based on the HTTP specification and can be easily implemented by developers.

The solution is divided in two concepts:

- A specification for allowing external applications to access the test parameters that will be used to test the microservice. This specification consists in a set of contents with the necessary information to compose the requests that will be issued in tests.
- A mechanism to attach and expose the specification. This includes how the specification is defined in the application side and how an external application can obtain this specification and further use it to run the performance test.

The key for this approach is to attach the specification to the microservice. This way we have each service operation with its own specification, test boundaries are well defined across the application and there is a mean to centralize all performance tests within one application, i.e., the test application. Additionally, we ensure that the test specification is consistent with each new release. The architectural view of the solution is represented in Figure 2.

We have chosen to provide the test specification as a response to a request that uses the HTTP Options method in a target URI. The target URI for obtaining test specifications is the very same through which the service provides its resources/operations.

The Options method is often used as a way for the system to provide information about its resources [3]. The kind of information may include available requests/responses, requirements and server capabilities. Therefore, the proposed solution use a method that

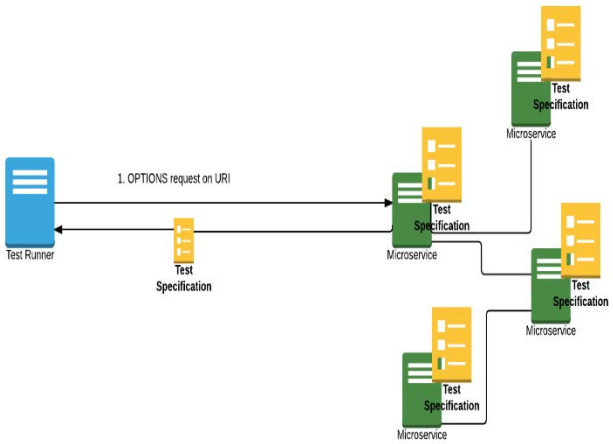

Figure 2. Architectural View of the Proposed Solution

does not affect the regular ones already provided by the service, such as POST and GET. This kind of separation will allow external applications to get the test specification and work in an automated manner.

The specification will provide to the test application (a.k.a. Test Runner) a document that contains: all methods that are available in


```
{
  "POST":{
    "description":"Save a new financial transaction",
    "parameter":"{
      "name":"000023989",
      "country":"766",
      "acctMonth":"10",
      "acctYear":"2015",
      "accountLines":  [{
        "amountUsd":1000,
        "amountPlan":1000.0,
        "account":"0083289"}]}",
    "parameter-schema":"{
      "type":"object",
      "id":"urn:jsonschema:com:finance:company:journal:bean:
            FinancialTransaction",
      "properties":{
        "id":{"type":"string"},
        "name":{"type":"string"},
        "country":{"type":"string"},
        "acctMonth":{"type":"string"},
        "acctYear":{"type":"string},
        "accountLines":{"type":"array","items":{"type":"object",
        "id":"urn:jsonschema:com:finance:bean:AccountDetails",
        "properties":{"id":{"type":"string"},
        "amountUsd":{"type":"number"},
        "amountPlan":{"type":"number"},
        "account":{"type":"string"}}}}}}",
    "validation":"{"headerData":{"status":"200"}}"
  },
  "GET":{
    "description":"Get transaction by id",
    "parameter":"{"parameter":"id","value":"1"}",
    "validation":"{"bodyData":{"name":"000023989",
      "country":"766","acctMonth":"10","acctYear":"2015"},
      "headerData":{"status":"200"}}"
  }
}
```


Figure 3. Example of a Response for an OPTION Request

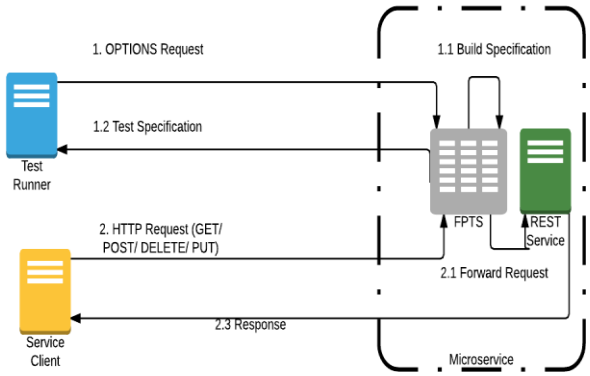

Figure 4. Framework Behavior for HTTP Requests

the requested path, sample data to be used as request parameter, a description that gives a semantic meaning for the method and validation data that can be used to validate the response.

Besides the parameter, the service will also provide a JSON Schema [7], which will allow the customization of the test parameters.

Let us say that the architecture specified by Figure 2 is implemented in some financial system and one of the microservices provides operations over financial transactions. A financial transaction has an 'ID', 'Country', 'Name', 'Month' and 'Year'. Besides that, it also contains a list of account details each one has an 'Account' and two attributes for amount values: 'AmountUsd' and 'AmountPln'. This particular microservice has two operations: find a transaction by its ID, and save a new transaction. Both operations can be requested by sending, respectively, GET and POST HTTP requests to the following URI: /rest/finance.

As this microservice is designed to allow automating performance tests, a test application can get the test specification to perform such test by sending an OPTIONS request to '/rest/finance'. For this particular request, we have a response such as the one represented by Figure 3.

With this proposal, we want to be able to provide a clear and simple specification to the test application. Moreover, we want the test application to find enough subsidies to execute performance tests on the target URI for all operations that the service accepts in this URI. This includes the parameter to use in the request and rules to validate the response. These validations allow the execution of some functional tests along with the performance tests.

5. FRAMEWORK

To validate the proposed architecture, we have developed an implementation of a framework for performance test specification (a.k.a. FPTS) in Java using the Spring Boot framework [6]. Spring Boot offers an embedded Tomcat or Jetty server along with Web, MVC and Persistence frameworks, all set up with minimal configuration overhead, which makes it appropriate for hosting and developing microservices [5,6].

The key concept for our framework operation is to work as a filter for HTTP requests to the microservice. Therefore, our framework will check each request that is sent to the service, and in case of an OPTIONS request the framework generates the specification present in the service and returns the full test specification for the requested URI. As the requested URI may allow different HTTP methods, the result response can contain more than one test specification (see Figure 3 as an example).

As one can see, the framework works as a layer before the service itself. By filtering each messages that comes to the service the framework is capable to respond to OPTIONS requests with the test specification and for all other requests the framework will forward the request to the service. This mean that the regular operations that the service provide will not be affected by the framework behavior.

5.1 Implementation

We focused our implementation in facilitating the usage and at the same time, we also want to keep the framework as transparent as possible allowing the developer the option to include it or remove it during the build process of the service. With that in mind, we used Java annotations to give the developer a clean and easy option to build the methods that will return the test specifications. On the other hand, we used the build engine Maven to split the framework into two artifacts: one for the API and one with the implementation.

In the API we placed the annotations and the classes to build the specification itself. The implementation artifact contains the filter that will handle the requests to the service as well as main classes that handles the process of building the test specification based on the annotation properties and the object returned by the annotated method. This separation allows the developer to compose build directives and choose to include the framework implementation or not without any code change needed. During runtime, the API will look for the provider class that is present in the implementation artifact, if the implementation is not found the framework capabilities will be turned off and the service will operate without the framework.

To keep the framework as independent as possible, we used reflection to extract the specification from the service and compose the response. This way, the framework keeps no direct reference or dependency to the service. This also allows the framework to work in a seamless way without interfering in other requests. This implementation also makes possible the removal of the framework without the necessity of a code change which may be useful when deploying the application to production or other controlled environments.

5.2 Usage

From the developer viewpoint, the framework allows microservices developers to expose test specifications with minimum overhead, by using Java annotations in methods that will return the specification. The first thing to do to start using the annotations and the framework classes is to add the API library to the service. After that, the developer has to implement the specification methods, one specification method for each service operation that will be exposed for performance testing. The final step is to add the framework filter to the list of web filters; this will make sure that the framework will be called on each request to the microservice.

Let us consider the same example that was described in the previous section (see Figure 2). Now, we want to expose a test specification for the save method. From our example, this method is accessed sending a POST request to the '/rest/finance' endpoint. To achieve this we can create a new method in the service class and annotate it with the *@PerformanceTest* annotation, as shown in Figure 5.

The Java annotation is the mean for the framework to find out which methods will be exposed for performance testing and also the specification object (TestSpec) that will be used by the test application. The object returned by the method invocation contains the parameters that will be used for service invocation. In the provided sample, the return will be a JSON object of type 'FinancialTransaction'. Besides the parameter, the 'TestSpec' will also store the validation data. In the given example, we are

```
@PerformanceTest(path = "/rest/finance",
   httpMethod = HttpMethodEnum.POST,
   description = "Save a new financial
   transaction")
public TestSpec<FinancialTransaction>
getTestSpecForSave() throws IOException
{
   FinancialTransaction testObj =
      new FinancialTransaction("000023989",
            "766", "10", "2015");
   AccountDetails accDetails =
      AccountDetails.build("0083289", 1000d,
         1000d);
   testObj.setAccLines(accDetails);
   TestValidationsBuilder validationBuilder =
      new TestValidationsBuilder();
   TestSpec<FinancialTransaction> testSpec =
      new TestSpec<FinancialTransaction>
         (testObj, validationBuilder.
           buildHeaderStatus200AndEntityBody(
           testObj));
   return testSpec;
}
```

Figure 5. Framework Usage to Expose a Specification

validating only the header status parameter to check if its value is 200 (OK).

The annotation holds the path in which the service provide its operation, a description and the HTTP method used in this operation. This data along with the 'TestSpec' object will compose the full specification (Figure 3). In the case where the service provides more than one operation in the same path, the framework will generate the specification for each operation and append them in one response (Figure 3).

Please note that we placed the attributes values directly on the code to show the link between the attributes values and the generated specification that is represented in Figure 3 as the POST object. In order to enhance the test capabilities it is recommended to use dynamic values that can be obtained from a database.

5.3 Key Features

Our goal with this framework is to provide a clean and easy way to expose the specification for the performance test. The result achieved with this framework was very promising; we were able to add some useful features such as:

- *Framework API:* This allows the developer to remove the framework from the application during the build. This is particularly useful when we are building the project for production deployment as we may not want to expose the test methods.
- *Annotation capabilities:* This provide a clean and simple way for the developer to define the specifications. In addition, this facility allows the developer to choose which service methods will be exposed.
- *Dynamic data for testing:* This refers to the possibility of using the database to fetch data for the test specification and validation. With this, the developer can bypass the changes that the data may have which could lead to failure in the tests. Let us say that we want to test a GET operation and we will get some data by its ID. Therefore, we can query the database and get the object with ID = 1 and provide the same object for validation in the specification.

```
Caused by:
org.apache.maven.plugin.compiler.CompilationFailureException:
Compilation failure
Returning type must have the generic class. Please set:
TestSpec<Class> as the returning type at
org.apache.maven.plugin.compiler.AbstractCompilerMojo.execute(Ab
stractCompilerMojo.java:858)
      at
org.apache.maven.plugin.compiler.CompilerMojo.execute(CompilerMo
jo.java:129)
      at
org.apache.maven.plugin.DefaultBuildPluginManager.executeMojo(De
faultBuildPluginManager.java:134)
      at
org.apache.maven.lifecycle.internal.MojoExecutor.execute(MojoExe
cutor.java:208)
      ... 20 more
```

Figure 6. Compile Error for Invalid Return Type

Although this object may change, the framework will always build the updated data and the test application will have the refreshed data to use.

- *Annotation processor:* This feature checks if the return type for the annotated method is valid. This check happens during compile time, which prevents errors in runtime and allows the developer to fix these inconsistencies quickly. Figure 4 presents a correct method. If the returning class of the annotated method has a different type or if it does not have the generic parameter, the compiler will throw an error. If we set the returning type as 'TestSpec' without the generic class, the error presented in Figure 6 is thrown.

- *Build features for validation data:* common validation types have been created in order to append the validation data to the specification, such as: validate only header status and validate for both header and body. This provides a quick way for the developer to define the validations rules and add some functional test statements to make the performance tests more reliable.

6. EVALUATION

As part of the evaluation process, we set up a financial application, in two scenarios: with the framework and without the framework. Our goal with this evaluation is to check the impact of the framework usage in the application's performance since the framework captures all requests even the ones that are not OPTIONS requests.

By doing the evaluation, we want to check if the framework will affect in the service performance and potentially influence the test performance results.

6.1 Methodology

For this evaluation, we will consider the number of samples as the total amount of requests. To discover the ideal number of samples to use we performed a pilot test to obtain the standard deviation mean, for this particular test, we used the application with the framework and this was decided in a random trial. The pilot test load configuration was 20 threads with 50 requests each, resulting in a total of 1,000 requests and the result of this test is shown in Figure 7.

For the response time the result was an average of 226ms and the deviation was 275 for this pilot test. Also, we used a confidence level of 99% which gave us a z value of 1.96 and a margin of error of 0.5%. With these values, we applied the following formula to discover the sample size needed:

$$\text{Necessary Sample Size} = \frac{Z \quad * \quad StdDev}{(\text{margin of error})}$$

The final result for the necessary sample size was approximate to 10,000. We divided the requests (samples) into 25 threads sending 400 requests each, resulting in 10,000 requests. In addition, to overcome undesired variations such as network problems we ran two rounds of tests for each application and for comparison we took the mean of those two rounds. To define the test order, we ran a random trial with the two possible values: with the framework (WF) and without the framework (NF). The final order was WF, NF, NF, WF.

6.2 Configuration and Execution

We deployed both applications under Amazon EC2 using an instance of type t1 micro (1 vCPU, 3.75GB memory and SSD storage). Each test round was executed with only one of the applications running each time. Besides that, the virtual machine was dedicated to run the application, no other process besides the ones from the operational system were executing.

For this evaluation, we have focused on two parameters: throughput and response time. The performance tests were executed using the JMeter tool [14].

The test procedure consisted of a POST request using a 'FinancialTransaction' object. The object is the very same as the one presented in Figure 5 and identified as the 'testObj' variable. On the microservice side, the object is inserted into an in-memory database to avoid bottlenecks on the service side, the same object with the ID attribute (auto generated by the service) is returned as response.

6.3 Results

The results for the tests with the framework are presented in Table 1 in the order that they were performed. Table 2 contains the mean for each case (WF and NF).

Based on the results, we cannot assert that the framework caused any impact on the application's performance. As we can see, the average response time mean was less than the one found in the application without the framework. From the implementation these results were expected because for requests that does not use OPTIONS we only forward the request to the service, no processing is made in the framework side other than the HTTP method check.

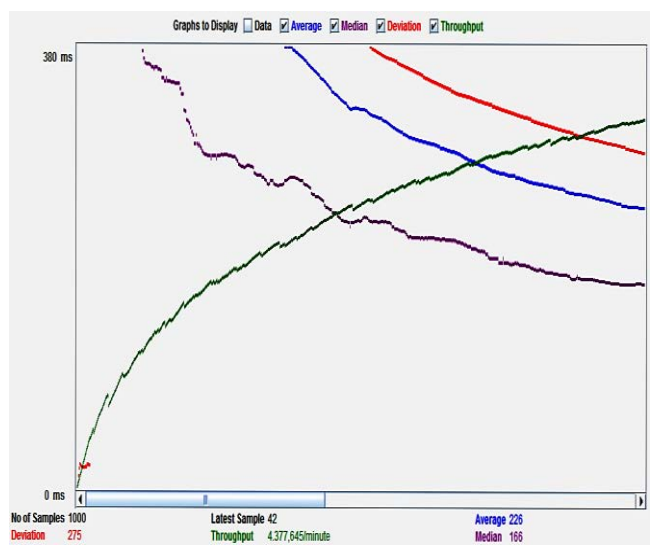

Figure 7. – Results for the Pilot Test

Table 1 – Test results for the four rounds

Test round	Test Results for each round	
	Avg. Response Time (seconds)	*Avg. Throughput (per second)*
1 – WF	117	50.64
2 – NF	122	48.66
3 - NF	119	49.17
4 - WF	119	50.07

Table 2 - Throughput and response time mean for each case

Test round	Mean for the values obtained after the rounds	
	Response Time Mean (seconds)	*Throughput Mean (per second)*
1 – WF	118	50.35
2 - NF	120.5	48.91

Additionally, the absence of impact in the service performance allow it to be deployed in any environment with the framework appended to it. The framework could be used to track down performance issues in production environment for example. In addition, the service provider could use the framework to provide a mean for the clients to perform the test and ensure that they are getting the minimum QoS parameters that they hired. This can be used to give a guarantee and reliability to the service clients as they can attest by themselves with minimal effort using the specification provided by the service.

Overall, the evaluation results are promising and show that the framework can be used without restrictions or side effects.

7. CONCLUSION

Microservices emerged as an alternative to develop applications and overcome some of the challenges found in traditional monolithic applications. The decoupled nature of the services give more flexibility on the deployment, development, scalability and choice of technologies. This flexibility ends up increasing the complexity of some development activities such as testing. Focusing on performance tests in applications based on the microservices architecture it is noticeable that for a large set of services (e.g. 100) the manual control of each test specification (generally, will be one for each service operation) becomes a hard task.

In order to simplify the execution of tests, we have presented in this paper an architectural model to allow automating the execution of performance tests in applications that are based on the microservices architecture.

Our approach was developed to be easily integrated, demanding only small changes in the application. The main feature of our proposal is to embed the test specification in each service. This way we enforce that it will be consistent with the service implementation despite any changes that the service may have suffered. We also employed the OPTIONS HTTP method to expose the specification and allow that external testing applications are able to access it at runtime.

Along with the architecture, we have developed a framework that implements some of the planned features. We tested the framework in an evaluation study. The evaluation shows that the framework has no impact on service execution.

Overall, the result of this study is promising. The proposed architecture is simple and effective, providing a clean and easy way to automate the performance test on microservices. The developed framework is easy to use and integrate, and does not have impact on the performance of the service operations which allows it to be used in any environment and to collect realistic performance parameters.

As a next step, we intend to evolve the framework, providing a client side API to facilitate its integration with well-known performance testing tools such as JMeter. With this improvement, we can also develop a central repository to manage all performance tests in a single end. This repository can come with a web module that provides test runs in a quick manner. We can also provide a historical database to store test results. This would be interesting for analyzing the impact of changes in the lifetime of applications with large sets of microservices.

A good usage for the framework is to append a test execution library to the service. This would allow the execution of performance tests on the infrastructure where the service is executed. To achieve this, docker containers could be used to provide lightweight clients to send the test requests. The main advantage of this approach is to give a realistic performance result that will only depend on the service implementation and the infrastructure removing the network that may cause disruptions on the performance values.

Another improvement can be to increase the capabilities for functional tests. With this, we can provide a large set of validations that will allow the execution of functional tests using the same approach that we have adopted for performance tests.

Finally, we also intend to implement the framework using other languages, such as Go and Python. This is important because one of the premises of a microservices architecture is to be polyglot in terms of programming languages.

8. REFERENCES

[1] Rahman, Mazedur, and Jerry Gao. A Reusable Automated Acceptance Testing Architecture for Microservices in Behavior-Driven Development. *Service-Oriented System Engineering (SOSE), 2015 IEEE Symposium on*. IEEE, 2015.

[2] Namiot, Dmitry and Sneps-Sneppe, Manfred. On Micro-services Architecture. International Journal of Open Information Technologies, v. 2, n. 9, p. 24-27, 2014.

[3] Http Options Method. Last access: 2016-03-02. Available: https://www.w3.org/Protocols/rfc2616/rfc2616-sec9.html.

[4] Fowler, Martin, LEWIS, James. Last access: 2016-03-15. microservices. Available: http://martinfowler.com/articles/ microservices.html.

[5] Building a Microservice Architecture with Spring Boot and Docker, part I. Last access: 2016-03-25. Available: http://www.3pillarglobal.com/insights/building-a-micro service-architecture-with-spring-boot-and-docker-part-i.

[6] Microservices with Spring. Last access: 2016-04-25. Available: https://spring.io/blog/2015/07/14/microservices-with-spring.

[7] JSON-Schema Specification. Last access: 2016-04-04. Available: http://tools.ietf.org/html/draft-zyp-json-schema-04.

[8] Scaling microservices at Gilt. Last access: 2016-04-12. Available: http://www.infoq.com/news/2015/04/scaling-microservices-gilt.

[9] Clark, Kim. Microservices, SOA, and APIs: Friends or enemies?. Last access: 2016-05-15. Available: http://www.ibm.com/developerworks/websphere/library/techarticles/1601_clark-trs/1601_clark.html?ca=drs-&cm_mmc=dw-_-trs-_-social-_-generic.

[10] What are microservices?. Last access: 2016-03-19. Available: https://opensource.com/resources/what-are-microservices.

[11] Amaral, Marcelo, et al. "Performance Evaluation of microservices Architectures using Containers."*Network Computing and Applications (NCA), 2015 IEEE 14th International Symposium on*. IEEE, 2015.

[12] Newman, Sam. Building microservices. " O'Reilly Media, Inc.", 2015.

[13] Experience from failing with microservices. Last access: 2016-05-08. Available: http://www.infoq.com/news/2014/08/failing-microservices.

[14] Apache JMeter. Last access: 2016-03-19. Available : http://jmeter.apache.org/.

[15] Kao, Chia Hung, Chun Cheng Lin, and Juei-Nan Chen. "Performance testing framework for rest-based web applications." *Quality Software (QSIC), 2013 13th International Conference on*. IEEE, 2013.

[16] Myers, Glenford J., Corey Sandler, and Tom Badgett. *The art of software testing*. John Wiley & Sons, 2011.

[17] Types of Performance Testing. Last access: 2016-03-23. Available: https://msdn.microsoft.com/en-us/library/bb924357.aspx

[18] 7 Types of Web Performance Tests and How They Fit Into Your Testing Cycle. Last access: 2016-03-23. Available: http://blog.smartbear.com/software-quality/7-types-of-web-performance-tests-and-how-they-fit-into-your-testing-cycle/

[19] Thakare, Sheetal, Savita Chavan, and P. M. Chawan. "Software Testing Strategies and Techniques." *International Journal of Emerging Technology and Advanced Engineering* (2012).

[20] Sathawornwichit, Chaiwat, and Shigeru Hosono. "Consistent Testing framework for Continuous Service Development." *Proceedings of the 2014 Asia-Pacific Services Computing Conference*. IEEE Computer Society, 2014.

[21] Pahl, Claus, and Pooyan Jamshidi. "Microservices: A Systematic Mapping Study."